

Task based on Backend Session 1

Hey everyone 🙌

Here's a small practical task based on Backend Session 1.

The goal is to build a simple backend API using Node.js + Express and practice the basics we covered: servers, routes, HTTP methods, request/response, JSON body, GET, POST, and basic async/await thinking.

Task: Student API

Scenario:

You are building a very small backend service for a training center. The service should store student data in memory and expose endpoints that allow clients to read and add students.

Main requirements:

- Create an Express server.
- Use `express.json()` to read JSON request bodies.
- Store students in a simple in-memory array.
- Do not use MongoDB yet.
- Use proper HTTP methods: GET and POST.
- Return JSON responses.
- Add simple validation when creating a student.
- Test the API using Postman, Thunder Client, or the browser for GET requests.

Starter data:

Start with this array inside your server file:

```
const students = [  
  { id: 1, name: "Ahmad", age: 20 },  
  { id: 2, name: "Sara", age: 22 }  
];
```

Required endpoints:

1) GET /

Return a welcome message.

Expected response:

```
{
  "message": "Welcome to the Student API"
}
```

2) GET /students

Return the full list of students.

Expected response:

```
{
  "students": [
    { "id": 1, "name": "Ahmad", "age": 20 },
    { "id": 2, "name": "Sara", "age": 22 }
  ]
}
```

3) GET /students/:id

Return one student by ID.

Expected response:

```
{
  "student": {
    "id": 1,
    "name": "Ahmad",
    "age": 20
  }
}
```

If the student does not exist, return:

```
{
  "message": "Student not found"
}
```

4) POST /students

Add a new student using name and age from the request body.

Request body example:

```
{
  "name": "Mona",
  "age": 21
}
```

Expected response:

```
{
  "message": "Student added successfully",
  "student": {
    "id": 3,
    "name": "Mona",
    "age": 21
  }
}
```

Validation requirement:

When adding a new student using POST /students, check that both name and age exist.

If one of them is missing, return status 400 with:

```
{
  "message": "Name and age are required"
}
```

Bonus challenge:

5) DELETE /students/:id

Delete a student by ID.

Expected response:

```
{
  "message": "Student deleted successfully"
}
```

If the student does not exist, return:

```
{
  "message": "Student not found"
}
```